

SEGUIMIENTO IV

Sara Rondon, Natalia Ángel
Electrónica digital II

Índice

1. Identificación del proyecto	2
2. Resumen	2
3. Descripción del hardware	2
4. Descripción del software	3
5. Estructura del código	3
6. Explicación de funciones principales	3
7. Comunicación (si aplica)	4
8. Interfaz de usuario (si aplica)	4
9. Manejo de errores y seguridad	4
10.Código fuente	4

1. Identificación del proyecto

- **Nombre del proyecto:** Juego Dodger en ESP32 con pantalla OLED
- **Integrantes del equipo:** Sara Rondon, Natalia Ángel
- **Asignatura:** Electrónica Digital II

2. Resumen

El proyecto consiste en el desarrollo de un videojuego embebido tipo *Dodger*, implementado sobre un microcontrolador ESP32 y una pantalla OLED SSD1306 de 128×64 píxeles, programado en MicroPython.

El jugador controla un sprite con movimiento vertical cuyo objetivo es esquivar obstáculos que se desplazan horizontalmente por la pantalla. El sistema incluye una máquina de estados que gestiona los modos de menú, juego, pausa y finalización. Se implementan tres modos de dificultad (Clásico, Contra-tiempo y Hardcore), cada uno con parámetros diferenciados de velocidad y frecuencia de aparición de obstáculos.

El proyecto integra comunicación I2C, manejo de GPIO, modulación PWM para retroalimentación sonora, temporización precisa y diseño de software orientado a objetos.

3. Descripción del hardware

El sistema está basado en el microcontrolador ESP32, encargado del procesamiento del juego, la lectura de entradas y el control de salidas visuales y sonoras.

Entradas

- Pulsador UP (GPIO 32): movimiento del jugador hacia arriba.
- Pulsador DOWN (GPIO 33): movimiento del jugador hacia abajo.
- Pulsador START/PAUSE (GPIO 25): inicio, pausa y reanudación del juego.

Procesamiento

- Comunicación I2C para el control de la pantalla OLED.
- Lógica de juego basada en máquina de estados.
- Generación de obstáculos, detección de colisiones y cálculo de puntaje.

Salidas

- Pantalla OLED SSD1306 (128×64 píxeles).
- Buzzer pasivo controlado por PWM (GPIO 26).

El sistema es alimentado mediante el puerto USB del ESP32.

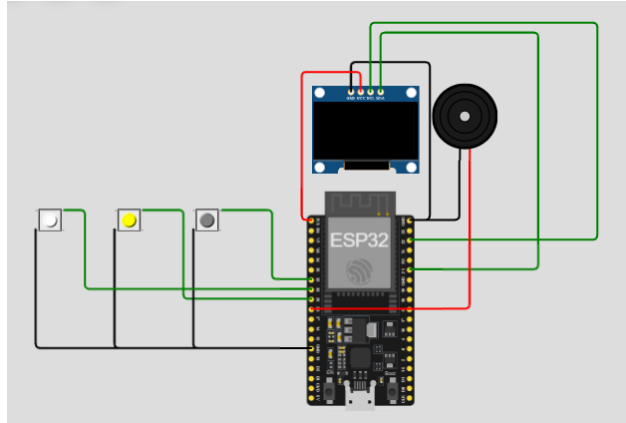


Figura 1: Diagrama de conexión del sistema

4. Descripción del software

El software está desarrollado en MicroPython, utilizando los módulos `machine`, `ssd1306`, `framebuf`, `utime` y `random`.

El programa implementa programación orientada a objetos para el manejo del jugador, los obstáculos y la dificultad del juego. Se utiliza una máquina de estados para separar claramente las diferentes fases del sistema: menú, juego, pausa y fin de partida.

El renderizado gráfico se realiza mediante el framebuffer de la pantalla OLED y la temporización del sistema se controla mediante un bucle principal sincronizado a una frecuencia fija de actualización.

5. Estructura del código

El código se encuentra organizado en los siguientes bloques:

- Configuración de pines y parámetros globales.
- Inicialización del hardware (OLED, botones y buzzer).
- Definición de sprites y framebuffers.
- Clases para jugador, obstáculos y dificultad.
- Máquina de estados del juego.
- Bucle principal de ejecución.

6. Explicación de funciones principales

- `buzzer_tone()`: Genera señales de audio mediante PWM como retroalimentación sonora.

- `DebounceButton.update()`: Implementa antirrebote por software para los pulsadores.
- `aabb()`: Realiza la detección de colisiones mediante bounding boxes.
- `Player.move_up()` / `move_down()`: Controlan el desplazamiento vertical del jugador.
- `Obstacle.update()`: Actualiza la posición y velocidad de los obstáculos.
- `Game.update()`: Controla la lógica principal del juego y las condiciones de finalización.
- `Game.draw()`: Renderiza los elementos gráficos según el estado del sistema.

7. Comunicación (si aplica)

La comunicación del sistema se realiza mediante el bus I2C, utilizado para el control de la pantalla OLED SSD1306. No se implementa comunicación serial, ya que toda la interacción con el usuario se realiza mediante pulsadores físicos.

8. Interfaz de usuario (si aplica)

La interfaz de usuario es gráfica y se despliega en la pantalla OLED. Incluye un menú de selección de modo de juego, un HUD durante la partida con puntaje, tiempo de supervivencia y obstáculos esquivados, además de pantallas de pausa y fin de juego.

La interacción se realiza a través de tres pulsadores, proporcionando una experiencia sencilla e intuitiva.

9. Manejo de errores y seguridad

El sistema emplea bloques `try/except` para manejar interrupciones inesperadas, garantizando la desactivación del buzzer y la limpieza de la pantalla.

El uso de antirrebote por software evita lecturas erróneas en los pulsadores, mejorando la estabilidad y robustez del sistema durante la ejecución.

10. Código fuente

```

1 from machine import Pin, I2C, PWM
2 import framebuf
3 import ssd1306
4 import utime
5 import random
6
7 # ----- CONFIGURACION -----
8 PIN_SDA = 21
9 PIN_SCL = 22

```

```

10 PIN_BTN_UP = 32
11 PIN_BTN_DOWN = 33
12 PIN_BTN_START = 25
13 PIN_BUZZ = 26 # buzzer pasivo
14
15 OLED_W, OLED_H = 128, 64
16 FPS = 30
17 FRAME_MS = int(1000 / FPS)
18
19 PLAYER_W, PLAYER_H = 8, 8
20 PLAYER_X = 10
21 OB_W, OB_H = 8, 10
22
23 MODES = ["CL SICO", "CONTRA-TIEMPO", "HARDCORE"]
24 CONTRA_SECONDS = 60 # duraci n del modo contra-tiempo
25
26 # ----- HARDWARE -----
27 i2c = I2C(0, scl=Pin(PIN_SCL), sda=Pin(PIN_SDA), freq=400000)
28 oled = ssd1306.SSD1306_I2C(OLED_W, OLED_H, i2c, addr=0x3C)
29
30 # Parche OLED
31 if not hasattr(oled, "hline"):
32     def hline(x, y, w, color):
33         for i in range(w):
34             oled.pixel(x + i, y, color)
35     oled.hline = hline
36
37 if not hasattr(oled, "vline"):
38     def vline(x, y, h, color):
39         for i in range(h):
40             oled.pixel(x, y + i, color)
41     oled.vline = vline
42
43 if not hasattr(oled, "blit"):
44     def blit(fb, x0, y0):
45         try:
46             w = fb.width
47             h = fb.height
48         except AttributeError:
49             w = 8
50             h = 8
51         for iy in range(h):
52             for ix in range(w):
53                 color = fb.pixel(ix, iy)
54                 if color:
55                     oled.pixel(x0 + ix, y0 + iy, 1)
56     oled.blit = blit
57
58 # ----- BOTONES -----
59 btn_up = Pin(PIN_BTN_UP, Pin.IN, Pin.PULL_UP)
60 btn_down = Pin(PIN_BTN_DOWN, Pin.IN, Pin.PULL_UP)
61 btn_start = Pin(PIN_BTN_START, Pin.IN, Pin.PULL_UP)
62
63 # ----- BUZZER -----

```

```

64 buzzer_pwm = PWM(Pin(PIN_BUZZ))
65 buzzer_pwm.duty(0)
66
67 def buzzer_tone(freq, dur_ms, duty=512):
68     buzzer_pwm.freq(int(freq))
69     buzzer_pwm.duty(int(duty))
70     utime.sleep_ms(dur_ms)
71     buzzer_pwm.duty(0)
72
73 def buzzer_async(freq, duty=400):
74     buzzer_pwm.freq(int(freq))
75     buzzer_pwm.duty(int(duty))
76
77 def buzzer_stop():
78     buzzer_pwm.duty(0)
79
80 # ----- SPRITE -----
81 SPRITE = bytearray([
82     0b00111100,
83     0b01111110,
84     0b11111111,
85     0b11011011,
86     0b11111111,
87     0b01111110,
88     0b00111100,
89     0b00011000
90 ])
91 fb_player = framebuffer.FrameBuffer(SPRITE, PLAYER_W, PLAYER_H, framebuffer.
    MONO_HMSB)
92
93 # ----- BOT N DEBOUNCE -----
94 class DebouncedButton:
95     def __init__(self, pin_obj, active_value=0, debounce_ms=40):
96         self.pin = pin_obj
97         self.active = active_value
98         self.debounce_ms = debounce_ms
99         self._last_state = self.pin.value()
100        self._last_time = utime.ticks_ms()
101        self._pressed_flag = False
102
103    def update(self):
104        v = self.pin.value()
105        now = utime.ticks_ms()
106        if v != self._last_state:
107            self._last_time = now
108            self._last_state = v
109        else:
110            if utime.ticks_diff(now, self._last_time) > self.debounce_ms:
111                if v == self.active and not self._pressed_flag:
112                    self._pressed_flag = True
113                    return True
114                if v != self.active and self._pressed_flag:
115                    self._pressed_flag = False
116            return False

```

```

117 btn_up_w = DebouncedButton(btn_up)
118 btn_down_w = DebouncedButton(btn_down)
119 btn_start_w = DebouncedButton(btn_start)
120
121
122 # ----- COLISION -----
123 def aabb(ax, ay, aw, ah, bx, by, bw, bh):
124     return not (ax+aw <= bx or bx+bw <= ax or ay+ah <= by or by+bh <= ay)
125
126 # ----- CLASES -----
127 class Player:
128     def __init__(self, y):
129         self.x = PLAYER_X
130         self.y = y
131         self.w = PLAYER_W
132         self.h = PLAYER_H
133
134     def move_up(self):
135         if self.y > 0:
136             self.y -= 8
137             buzzer_tone(1500, 30, 400)
138
139     def move_down(self):
140         if self.y + self.h < OLED_H:
141             self.y += 8
142             buzzer_tone(1400, 30, 400)
143
144     def draw(self, d):
145         d.blit(fb_player, int(self.x), int(self.y))
146
147 class Obstacle:
148     def __init__(self, x, y, w=OB_W, h=OB_H, speed=2):
149         self.x = x
150         self.y = y
151         self.w = w
152         self.h = h
153         self.speed = speed
154
155     def update(self):
156         self.x -= self.speed
157         buzzer_async(900 + random.randint(-100, 100), 300)
158
159     def offscreen(self):
160         return (self.x + self.w) < 0
161
162     def draw(self, d):
163         for yy in range(self.h):
164             d.hline(int(self.x), int(self.y)+yy, int(self.w), 1)
165
166 # ----- DIFICULTAD -----
167 class Difficulty:
168     def __init__(self, mode):
169         self.mode = mode
170         self.reset()

```

```

171
172     def reset(self):
173         if self.mode == 0:
174             self.spawn_ms = 1200
175             self.base_speed = 1.2
176             self.min_spawn = 700
177             self.step_time = 6000
178             self.speed_limit = 3.0
179             self.label = "CL SICO"
180         elif self.mode == 1:
181             self.spawn_ms = 800
182             self.base_speed = 2.2
183             self.min_spawn = 400
184             self.step_time = 3000
185             self.speed_limit = 3.5
186             self.label = "CONTRA"
187         else:
188             self.spawn_ms = 400
189             self.base_speed = 4.0
190             self.min_spawn = 150
191             self.step_time = 1200
192             self.speed_limit = 6.5
193             self.label = "HARDCORE"
194         self.last_tick = utime.ticks_ms()
195
196     def escalate(self):
197         now = utime.ticks_ms()
198         if utime.ticks_diff(now, self.last_tick) > self.step_time:
199             if self.spawn_ms > self.min_spawn:
200                 self.spawn_ms = max(self.min_spawn, self.spawn_ms - 30)
201             if self.base_speed < self.speed_limit:
202                 self.base_speed += 0.1
203             self.last_tick = now
204
205 # ----- ESTADOS -----
206 STATE_MENU = 0
207 STATE_GAME = 1
208 STATE_GAMEOVER = 2
209 STATE_PAUSE = 3 # NUEVO
210
211 class Game:
212     def __init__(self):
213         self.state = STATE_MENU
214         self.mode_idx = 0
215         self.player = Player((OLED_H - PLAYER_H)//2)
216         self.obstacles = []
217         self.last_spawn = utime.ticks_ms()
218         self.difficulty = Difficulty(self.mode_idx)
219         self.score = 0
220
221     # NUEVAS M TRICAS
222     self.start_time = 0
223     self.survival_time = 0
224     self.obstacles_dodged = 0

```



```

225
226     def start(self):
227         self.state = STATE_GAME
228         self.player = Player((OLED_H - PLAYER_H)//2)
229         self.obstacles = []
230         self.score = 0
231         self.last_spawn = utime.ticks_ms()
232         self.start_time = utime.ticks_ms()
233         self.obstacles_dodged = 0
234         self.difficulty = Difficulty(self.mode_idx)
235         buzzer_tone(1200, 150, 512)
236
237     def game_over(self):
238         self.state = STATE_GAMEOVER
239         buzzer_tone(300, 400, 700)
240         buzzer_stop()
241
242     def spawn(self):
243         y = random.randint(0, OLED_H - OB_H)
244         x = OLED_W
245         speed = int(self.difficulty.base_speed + random.random()*1.8)
246         self.obstacles.append(Obstacle(x, y, OB_W, OB_H, speed))
247
248     def update(self):
249         if self.state != STATE_GAME:
250             return
251
252         self.survival_time = (utime.ticks_ms() - self.start_time)//1000
253
254         self.difficulty.escalate()
255
256         now = utime.ticks_ms()
257         if utime.ticks_diff(now, self.last_spawn) > self.difficulty.
            spawn_ms:
258             self.spawn()
259             self.last_spawn = now
260
261         for ob in list(self.obstacles):
262             ob.update()
263             if ob.offscreen():
264                 self.obstacles.remove(ob)
265                 self.score += 5
266                 self.obstacles_dodged += 1 # NUEVO
267
268         for ob in self.obstacles:
269             if aabb(self.player.x, self.player.y, self.player.w, self.
                player.h,
270                 ob.x, ob.y, ob.w, ob.h):
271                 self.game_over()
272                 return
273
274         if self.mode_idx == 1:
275             elapsed = (utime.ticks_ms() - self.start_time) // 1000
276             if elapsed >= CONTRA_SECONDS:

```

```

277         self.game_over()
278
279     def draw(self, d):
280         d.fill(0)
281
282         if self.state == STATE_MENU:
283             d.text("DODGER", 36, 6, 1)
284             d.text("MOD0:", 6, 24, 1)
285             d.text(MODES[self.mode_idx], 46, 24, 1)
286             d.text("START -> JUGAR", 18, 44, 1)
287
288         elif self.state == STATE_GAME:
289             d.text("S:"+str(self.score), 0, 0, 1)
290             d.text("O:"+str(self.obstacles_dodged), 40, 0, 1) #
291             d.text("T:"+str(self.survival_time), 90, 0, 1) #
292             d.hline(0, 10, OLED_W, 1)
293
294             self.player.draw(d)
295             for ob in self.obstacles:
296                 ob.draw(d)
297
298         elif self.state == STATE_PAUSE:
299             d.text("PAUSA", 40, 20, 1)
300             d.text("S:"+str(self.score), 30, 40, 1)
301             d.text("START=CONT", 10, 55, 1)
302
303         elif self.state == STATE_GAMEOVER:
304             d.text("GAME OVER", 30, 10, 1)
305             d.text("P:"+str(self.score), 30, 25, 1)
306             d.text("T:"+str(self.survival_time), 30, 35, 1)
307             d.text("O:"+str(self.obstacles_dodged), 30, 45, 1)
308             d.text("START -> MENU", 5, 58, 1)
309
310         d.show()
311
312     # ----- LOOP -----
313     game = Game()
314
315     try:
316         while True:
317             if btn_up_w.update():
318                 if game.state == STATE_MENU:
319                     game.mode_idx = (game.mode_idx - 1) % len(MODES)
320                     buzzer_tone(1100, 40, 400)
321                 elif game.state == STATE_GAME:
322                     game.player.move_up()
323
324             if btn_down_w.update():
325                 if game.state == STATE_MENU:
326                     game.mode_idx = (game.mode_idx + 1) % len(MODES)
327                     buzzer_tone(900, 40, 400)
328                 elif game.state == STATE_GAME:
329                     game.player.move_down()
330

```

```

331         if btn_start_w.update():
332             if game.state == STATE_MENU:
333                 game.start()
334             elif game.state == STATE_GAME:
335                 game.state = STATE_PAUSE # PAUSA
336             elif game.state == STATE_PAUSE:
337                 game.state = STATE_GAME # REANUDA
338             elif game.state == STATE_GAMEOVER:
339                 game.state = STATE_MENU
340
341         game.update()
342         game.draw(oled)
343         utime.sleep_ms(FRAME_MS)
344
345     except KeyboardInterrupt:
346         buzzer_stop()
347         oled.fill(0)
348         oled.text("Fin del juego", 24, 28, 1)
349         oled.show()

```